CAMPUS EVENT SEATING AND ENTRY VALIDATION

Python Programming Case Study

1. Case Study

This case study validates a campus event seating plan against a list of registered tickets. Seats are stored in a nested list, where None indicates an empty seat. The program reports empty seats and duplicate seat numbers, records unique occupied seat numbers, and checks that each ticket appears among the registered seats.

2. Problem Statement

Event organizers need to find unassigned seats, prevent duplicate seat assignments, and verify that issued tickets correspond to registered seat numbers. Manual checking of a seating grid and ticket list can miss errors, so the program performs these checks systematically.

3. Objective

To scan a nested seating layout, report empty and duplicate seats, and flag ticket numbers that are not present in the validated seat list.

4. Assumptions

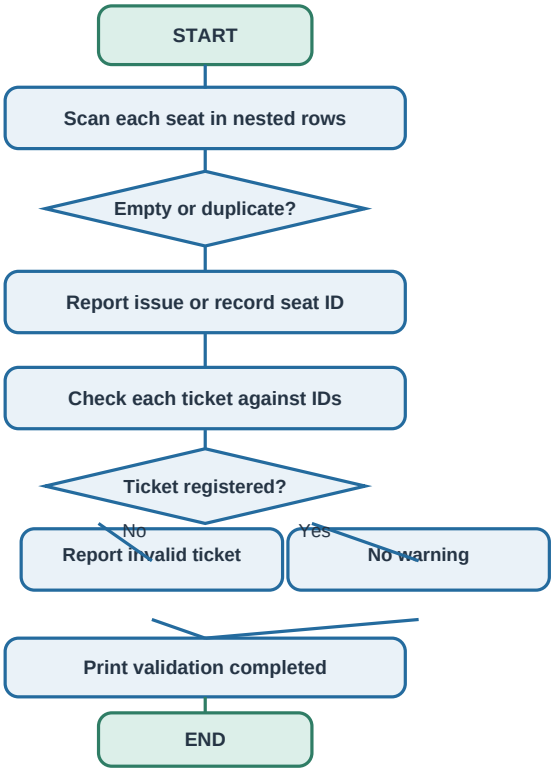
- Each row in the nested list represents one seating row.
- A seat contains an integer seat or ticket number, or None when empty.
- The registered list stores unique occupied seat numbers encountered during the scan.
- Tickets are valid only if their number is present in the registered seat list.
- Validation reports issues but does not alter seat assignments or ticket records.

5. Algorithm

1. Store the seat layout as a nested list and store the ticket numbers in a list.
2. Initialize an empty list of registered seat numbers.
3. Visit each seat in each row.
4. If a seat is None, report an empty seat; if its number is already registered, report a duplicate; otherwise, record it.
5. For each ticket, check whether its number appears in the registered list and report it if not.
6. Print that seating validation is completed.

6. Flowchart

The process validates seats first, then checks ticket numbers against the list of unique occupied seats.



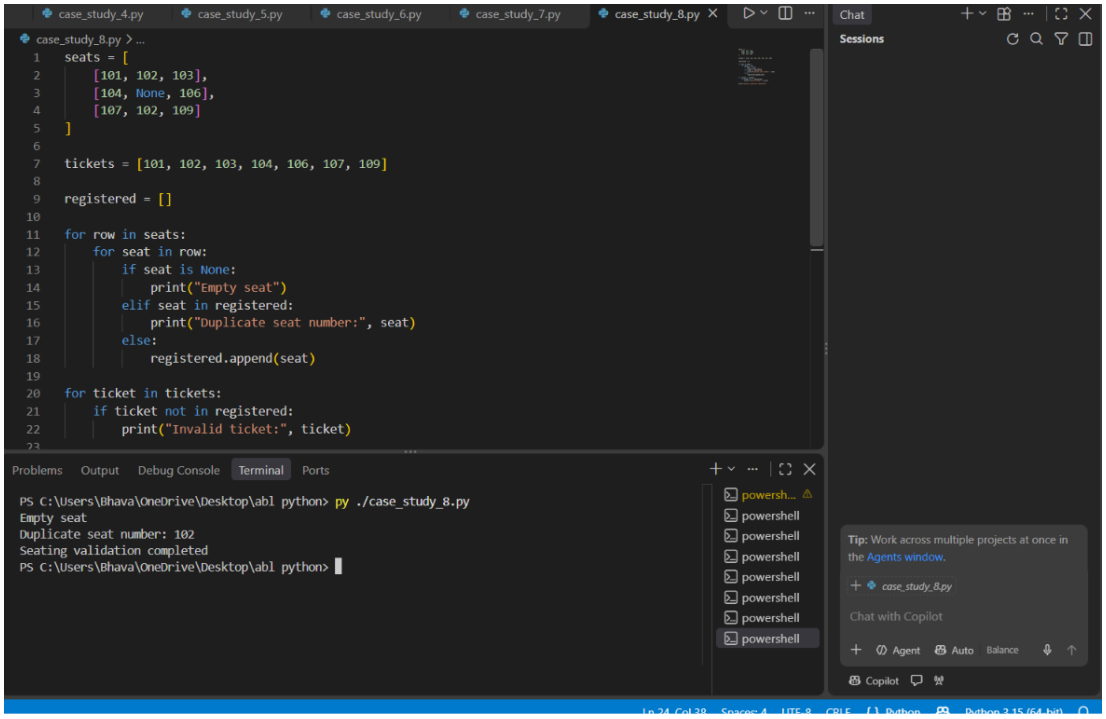
7. Pseudocode

```
BEGIN
  seats = nested seating list
  tickets = list of ticket numbers
  registered = empty list

  FOR each seat in each row
    IF seat is None: report empty seat
    ELSE IF seat is in registered
      report duplicate seat number
    ELSE: add seat to registered
  END FOR
  FOR each ticket in tickets
    IF ticket not in registered: report invalid ticket
  END FOR
  display validation completed
END
```

8. Program and Output Screenshot

The seating grid contains an empty seat and repeats seat number 102, so both are reported. The listed tickets are 101, 102, 103, 104, 106, 107, and 109; each appears in the occupied seats, so no invalid-ticket message is expected.



9. Conclusion

The validation checks seating availability, repeated seat numbers, and ticket-to-seat consistency. In the sample, one seat is empty, seat number 102 is duplicated, and all ticket numbers correspond to occupied seats. These checks can help event staff review seating and entry records before admitting attendees.

ENERGY CONSERVATION BEFORE-AND-AFTER ANALYSIS

Python Programming Case Study

1. Case Study

This case study compares energy use for three appliances before and after a conservation measure. For each appliance, the program finds the difference between before and after readings and expresses that difference as a percentage of the original use.

2. Problem Statement

It can be difficult to compare energy savings across appliances when their original energy use differs. A program can calculate the absolute and percentage savings for each appliance using the same formula.

3. Objective

To calculate energy saved and the corresponding savings percentage for each appliance from parallel before-and-after data lists.

4. Assumptions

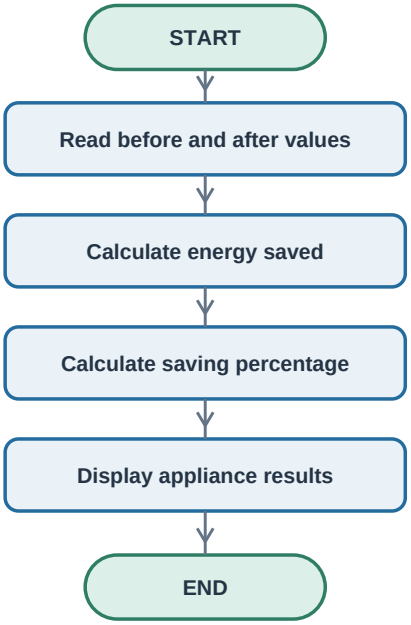
• The before and after lists contain the same number of readings and refer to the same appliances by index.
• Energy readings are numeric, non-negative, and use the same unit and measurement period.
• Each before reading is greater than zero so the percentage can be calculated.
• Energy saved is defined as before use minus after use; a negative result would indicate increased use.
• The reported saving percentage uses before use as its baseline.

5. Algorithm

1. Store the before and after energy readings in parallel lists.
2. Loop through each appliance index.
3. Calculate energy saved as before[i] minus after[i].
4. Calculate saving percentage as (energy saved / before[i]) × 100.
5. Display the appliance number, energy saved, and saving percentage.

6. Flowchart

The program compares each pair of readings, calculates both savings measures, and displays the results.



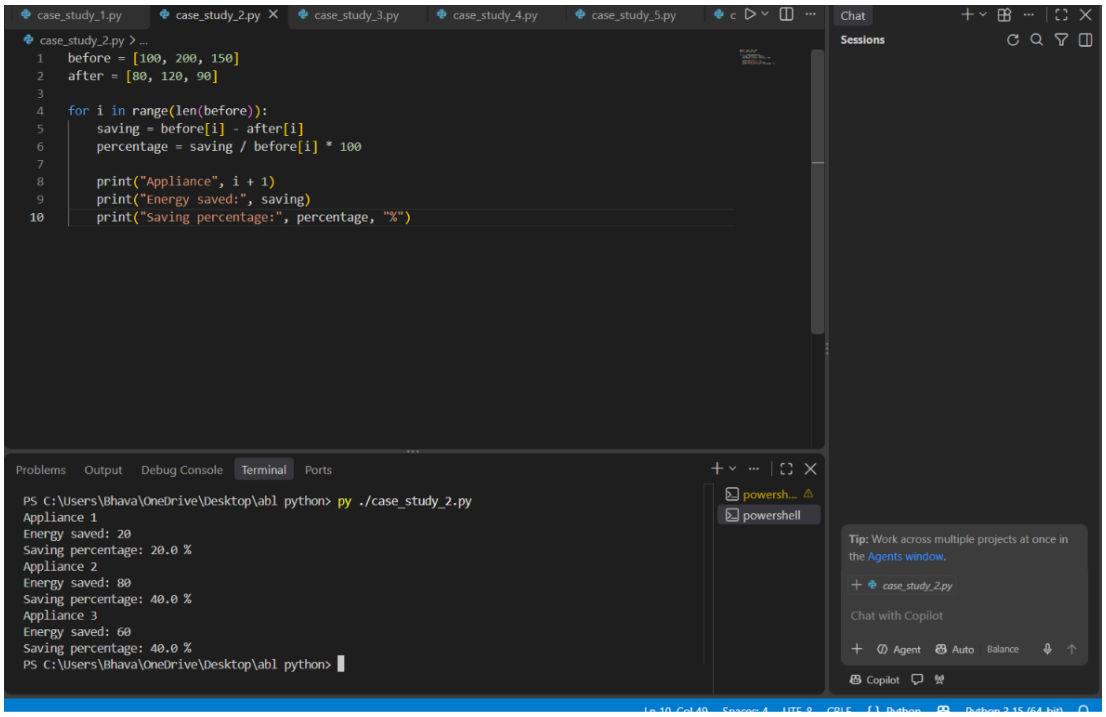
7. Pseudocode

```
BEGIN
  before = energy readings before conservation
  after = energy readings after conservation

  FOR each index i in before
    saving = before[i] - after[i]
    percentage = (saving / before[i]) * 100
    display appliance number i + 1
    display saving and percentage
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows before readings of 100, 200, and 150 and after readings of 80, 120, and 90. The calculated savings are 20 (20%), 80 (40%), and 60 (40%) for Appliances 1, 2, and 3.



9. Conclusion

The program calculates both the amount of energy saved and the percentage reduction for each appliance. In the example, Appliance 2 saves the largest amount, 80 units, while Appliances 2 and 3 share the highest percentage saving at 40%. This makes it easier to compare conservation results across different starting energy levels.

FOOD SUPPLY CHAIN STOCK RECONCILIATION

Python Programming Case Study

1. Case Study

This case study reconciles food stock at multiple locations. For each location, the program subtracts distributed quantities and damaged items from the quantity received, then compares the remaining stock with a minimum stock level of 20 units to determine whether replenishment is required.

2. Problem Statement

Food supply locations need to track usable stock after distribution and damage. Without reconciling these quantities, staff may overlook locations that need restocking or mistakenly order stock that remains sufficient.

3. Objective

To calculate remaining stock for each location and flag locations where remaining stock falls below the minimum threshold of 20 units.

4. Assumptions

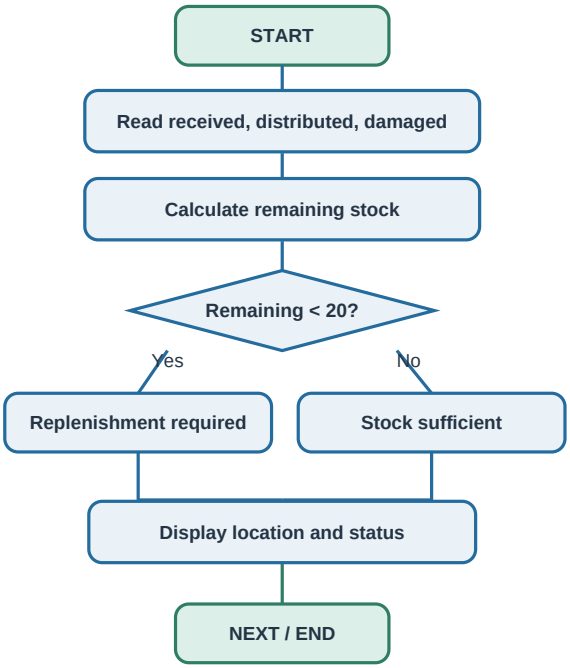
• The received, distributed, and damaged lists have the same length and corresponding indexes represent the same location.
• All values are non-negative whole quantities measured in the same unit.
• Damaged items are removed from usable stock in addition to distributed items.
• Replenishment is required only when remaining stock is less than 20; exactly 20 is considered sufficient.
• The program reports the need for replenishment but does not calculate an order quantity.

5. Algorithm

1. Store quantities received, distributed, and damaged for each location.
2. Set the minimum stock threshold to 20.
3. For each location, calculate remaining = received - distributed - damaged.
4. Display the location number and remaining stock.
5. If remaining stock is below 20, print “Replenishment required”; otherwise print “Stock is sufficient.”

6. Flowchart

For each location, the program reconciles usable stock and compares it with the minimum threshold.



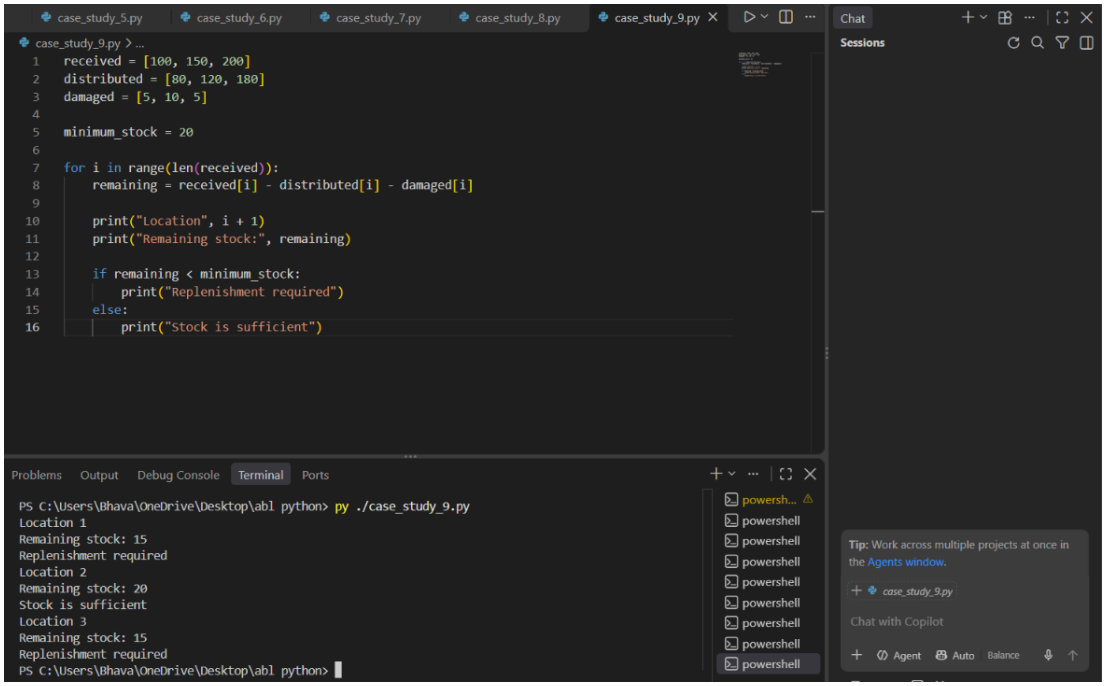
7. Pseudocode

```
BEGIN
  received, distributed, damaged = location quantities
  minimum_stock = 20

  FOR each location index i
    remaining = received[i] - distributed[i] - damaged[i]
    display location and remaining stock
    IF remaining < minimum_stock
      display replenishment required
    ELSE
      display stock is sufficient
    END IF
  END FOR
END
```

8. Program and Output Screenshot

The screenshot reports remaining stocks of 15 units at Location 1, 20 at Location 2, and 15 at Location 3. Locations 1 and 3 fall below the minimum stock threshold and require replenishment. Location 2 has exactly 20 units and is classified as sufficient.



9. Conclusion

The program calculates usable remaining stock after accounting for both distribution and damage. In the sample, two locations fall below the 20-unit threshold and are flagged for replenishment, while the location with exactly 20 units remains sufficient. This provides a basic stock-monitoring step for food supply operations.

MULTI-DAY BUS ROUTE PERFORMANCE ANALYSIS

Python Programming Case Study

1. Case Study

This case study summarizes passenger demand and fuel use for several bus routes over multiple days. For each route, the program calculates total passengers, average passengers per day, and checks whether the highest daily passenger count exceeds the bus capacity of 40.

2. Problem Statement

Transport planners need to review passenger demand and fuel use across routes. Aggregated totals and averages help compare activity, while checking the peak daily count against capacity can identify routes that may require a larger bus or another service.

3. Objective

To calculate route-level passenger totals and averages, display recorded fuel use, and flag a route when its maximum daily passenger count exceeds 40.

4. Assumptions

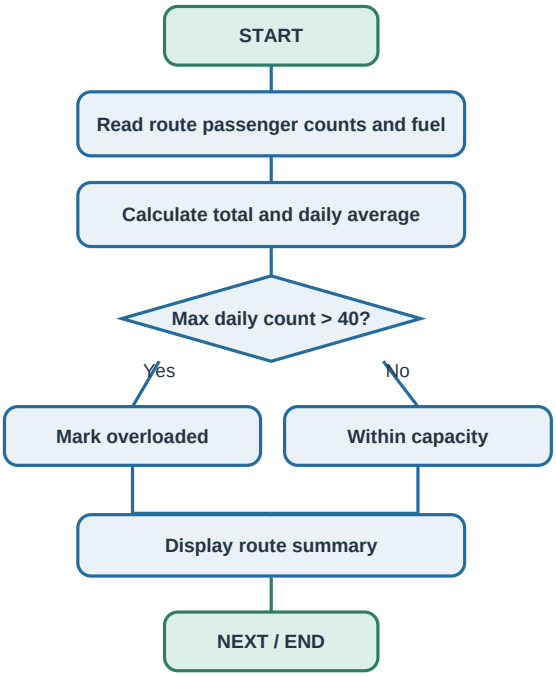
- Each row in the passengers list represents one route and contains daily passenger counts.
- The fuel list has one corresponding value per route; the program reports fuel but does not analyze it.
- Each route has at least one daily passenger count.
- Bus capacity is 40 passengers; a route is marked overloaded if any day's count is greater than 40.
- The input lists are aligned by route index.

5. Algorithm

1. Store daily passenger counts for each route, set capacity to 40, and store route fuel values.
2. Loop over each route.
3. Calculate the total passenger count and average across the route's days.
4. Display the route number, total, average, and fuel used.
5. If the maximum daily count exceeds 40, print "Overloaded route"; otherwise print "Route within capacity."

6. Flowchart

Each route is summarized and its peak daily passenger count is compared with the 40-passenger capacity.



7. Pseudocode

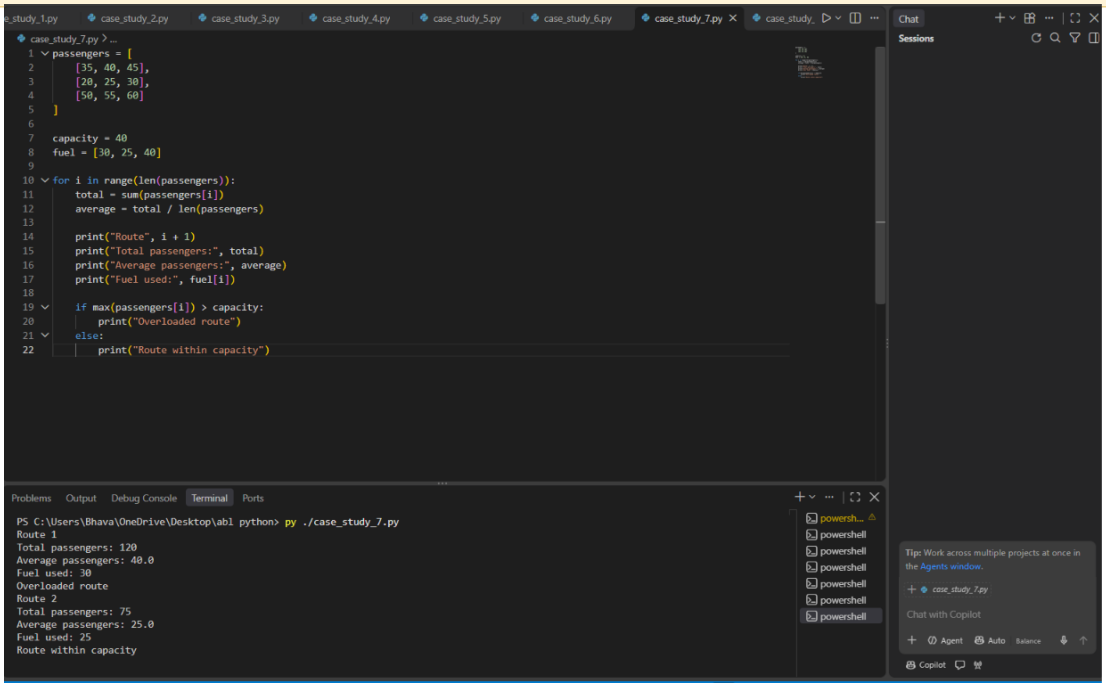
```
BEGIN
  passengers = daily counts by route
  capacity = 40; fuel = fuel values by route

  FOR each route index i
    total = sum(passengers[i])
    average = total / length(passengers[i])
    display route, total, average, fuel[i]
    IF max(passengers[i]) > capacity
      display overloaded route
    ELSE
      display route within capacity
    END IF
  END FOR
END
```

8. Program and Output Screenshot

The terminal panel shows output for Routes 1 and 2 only, although the source code includes three routes. For Route 3, the visible data [50, 55, 60] gives a total of 165, an average of 55, and a peak of 60, so the code's rule would classify it as overloaded.

> Route 3 values above are calculated from the source data; they are not visible in the screenshot terminal output.



9. Conclusion

The program combines passenger totals, averages, fuel values, and a peak-capacity check for each route. Route 1 is overloaded because its peak is 45, while Route 2 remains within the 40-passenger capacity with a peak of 30. From the visible Route 3 data, its peak is 60, so it would also be flagged as overloaded.

NESTED LIST SPORTS PERFORMANCE ANALYSIS

Python Programming Case Study

1. Case Study

This case study analyzes sports performance scores stored in a nested list. Each inner list contains one player's scores across several events. The program calculates each player's total and average score and labels performance as excellent when the total is greater than 100.

2. Problem Statement

Coaches may need to summarize multiple event scores for several players and identify high aggregate performance. Manually calculating totals and averages can be time-consuming. A program can perform the calculations consistently and apply a simple performance threshold.

3. Objective

To use a nested list to calculate each player's total and average score, display these statistics, and flag players whose total score exceeds 100.

4. Assumptions

• Each row in the nested list represents one player.
• Values in a row are numeric scores from the player's events and use a consistent scale.
• Each player has at least one score, so an average can be calculated.
• "Excellent performance" is printed only when the total is strictly greater than 100.
• The threshold is a sample rule and does not account for event difficulty or other performance measures.

5. Algorithm

1. Store the players' event scores in a nested list.
2. Loop through each player's score row.
3. Calculate the total using sum(row).
4. Calculate the average by dividing the total by the number of scores in the row.
5. Display the player number, total, and average.
6. If the total is greater than 100, display "Excellent performance."

6. Flowchart

For each player, the program computes the summary statistics and checks the total-score threshold.



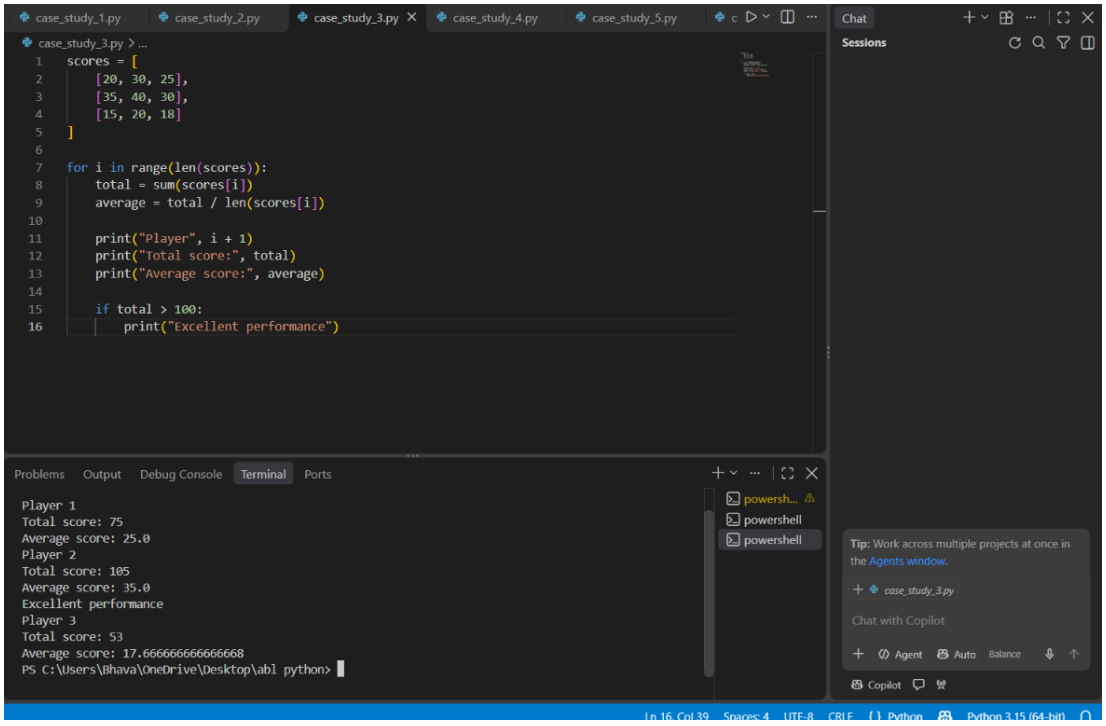
7. Pseudocode

```
BEGIN
  scores = nested list of player event scores

  FOR each player row at index i
    total = sum(scores[i])
    average = total / length(scores[i])
    display player number, total, average
    IF total > 100
      display excellent performance
    END IF
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows three players' scores. Player 1 totals 75 with an average of 25. Player 2 totals 105 with an average of 35 and is flagged for excellent performance. Player 3 totals 53 with an average of about 17.67.



9. Conclusion

The program summarizes each player's scores and applies a clear total-score condition. In the sample, Player 2 is the only player whose total exceeds 100, while the other players do not meet the excellent-performance threshold. The calculations make it easier to review player results across multiple events.

SMART CLASSROOM TIMETABLE ALLOCATION

Python Programming Case Study

1. Case Study

This case study allocates courses to classrooms and time periods while checking room capacity and schedule conflicts. For each course, the program compares enrollment with the selected room's capacity and checks whether that room-period pair has already been allocated.

2. Problem Statement

Classroom scheduling must account for both available seats and conflicting use of the same room at the same time. A room with insufficient capacity cannot accommodate the enrollment, and a room already assigned in a period cannot be used again for another class.

3. Objective

To validate each course allocation against room capacity and room-period availability, then report a capacity issue, timetable clash, or successful allocation.

4. Assumptions

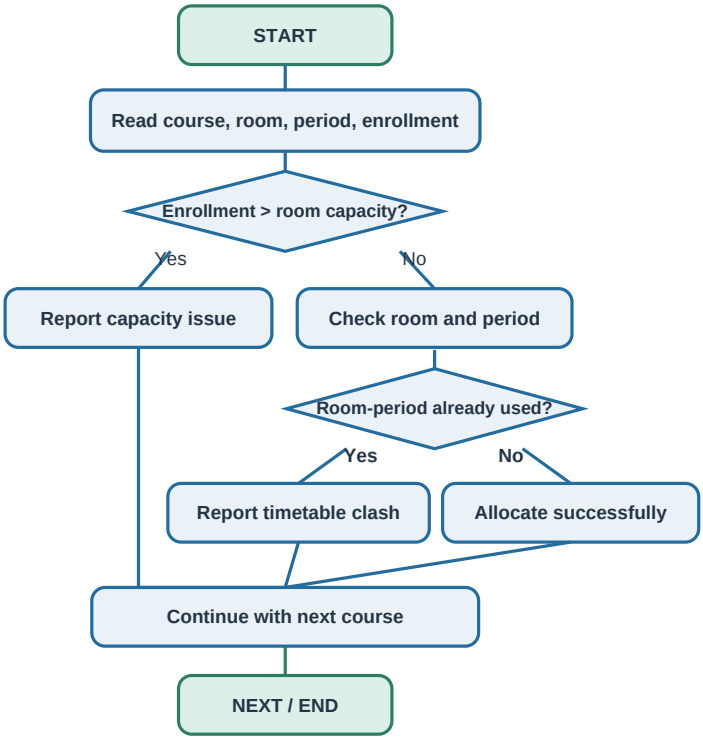
• Each course record contains a course name, room name, and period.
• Student enrollment and room capacity are available in dictionaries keyed by course and room.
• A room can host only one course in a given period; the same room can be reused in another period.
• Enrollment equal to room capacity is allowed; only enrollment greater than capacity is rejected.
• Capacity is checked before timetable conflict, so a course exceeding capacity is reported for that condition first.

5. Algorithm

1. Store course assignments, room capacities, and student enrollments.
2. Initialize an empty list of allocated room-period pairs.
3. For each course, display its course, room, and period.
4. If enrollment exceeds room capacity, report "Room capacity exceeded."
5. Otherwise, if the room-period pair is already allocated, report "Timetable clash."
6. Otherwise, add the room-period pair to the allocation list and report success.

6. Flowchart

For each course, the program checks room capacity first and then checks for a room-period conflict.



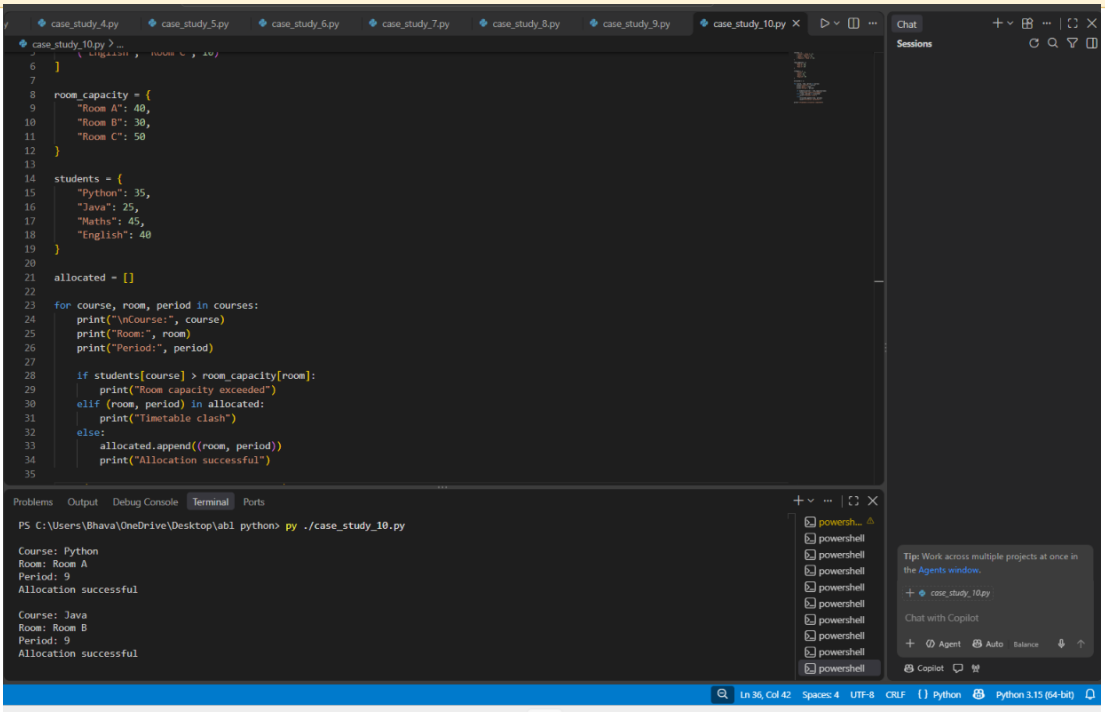
7. Pseudocode

```
BEGIN
  allocated = empty list
  FOR each (course, room, period)
    display course, room, period
    IF students[course] > room_capacity[room]
      display capacity exceeded
    ELSE IF (room, period) in allocated
      display timetable clash
    ELSE
      add (room, period) to allocated
      display allocation successful
    END IF
  END FOR
END
```

8. Program and Output Screenshot

The visible terminal output shows Python assigned to Room A in Period 9 and Java assigned to Room B in Period 9; both allocations are successful. The screenshot shows enrollment values and capacity limits, but the remaining course output is not visible.

The visible enrollments are Python 35, Java 25, Maths 45, and English 40. Room capacities are A: 40, B: 30, and C: 50. A course with enrollment 45 would exceed the capacity of Room A or Room B, but fits Room C.



9. Conclusion

The program applies two basic timetable checks: the room must have enough capacity, and the same room-period combination must not already be allocated. The visible Python and Java assignments pass these checks. The remaining course results cannot be confirmed from the cropped output shown in the screenshot.

SMART ELECTRICITY BILLING FOR MULTIPLE APARTMENTS

Python Programming Case Study

1. Case Study

This case study calculates electricity bills for multiple apartments from their recorded unit consumption. The program applies a tiered per-unit rate and a fixed charge, then prints the units and total bill for each apartment.

2. Problem Statement

Calculating bills for several apartments can be repetitive, especially when different consumption ranges use different rates. A program can apply a consistent tariff calculation to each apartment and show the resulting bill.

3. Objective

To calculate and display each apartment’s electricity bill using consumption tiers, per-unit rates, and a fixed charge.

4. Assumptions

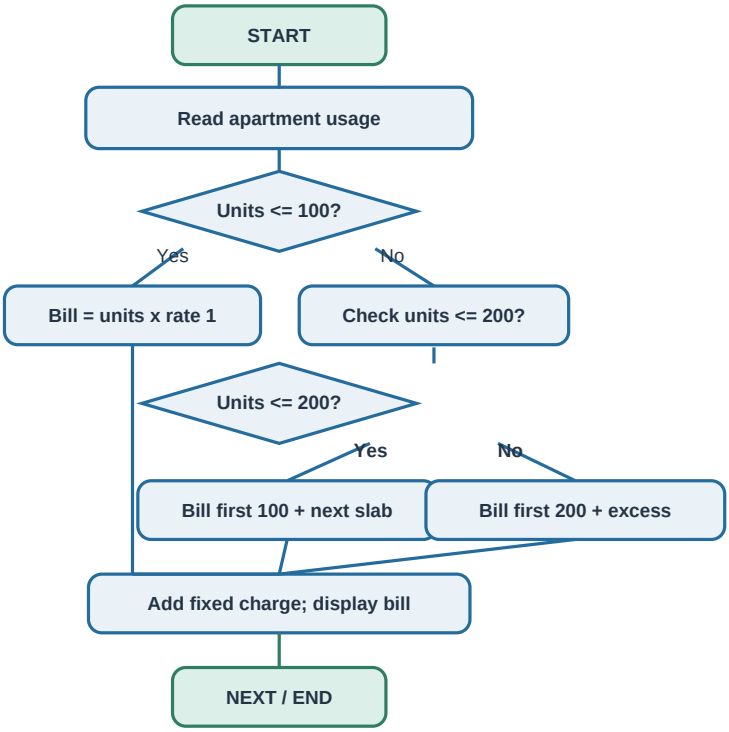
• Each value in the units list represents one apartment’s consumption for the billing period.
• Consumption is a non-negative whole number of units.
• The visible tariff uses rate1 = 2 for up to 100 units, rate2 = 3 for the next tier up to 200 units, and rate3 = 5 above 200 units.
• Billing is progressive by tier, as implied by the visible rate variables and branch structure.
• A fixed charge of 50 is added to each apartment bill.
• Currency is shown as Rs. in the source code.

5. Algorithm

1. Store apartment unit readings and define the three rates and fixed charge.
2. For each apartment, read its unit consumption.
3. If consumption is at most 100, multiply all units by rate1.
4. Otherwise, if consumption is at most 200, calculate the first 100 units at rate1 and remaining units at rate2.
5. Otherwise, calculate the first 100 units at rate1, the next 100 at rate2, and remaining units at rate3.
6. Add the fixed charge and display units and total bill.

6. Flowchart

The program selects a tariff calculation based on the apartment's unit range, adds the fixed charge, and displays the bill.



7. Pseudocode

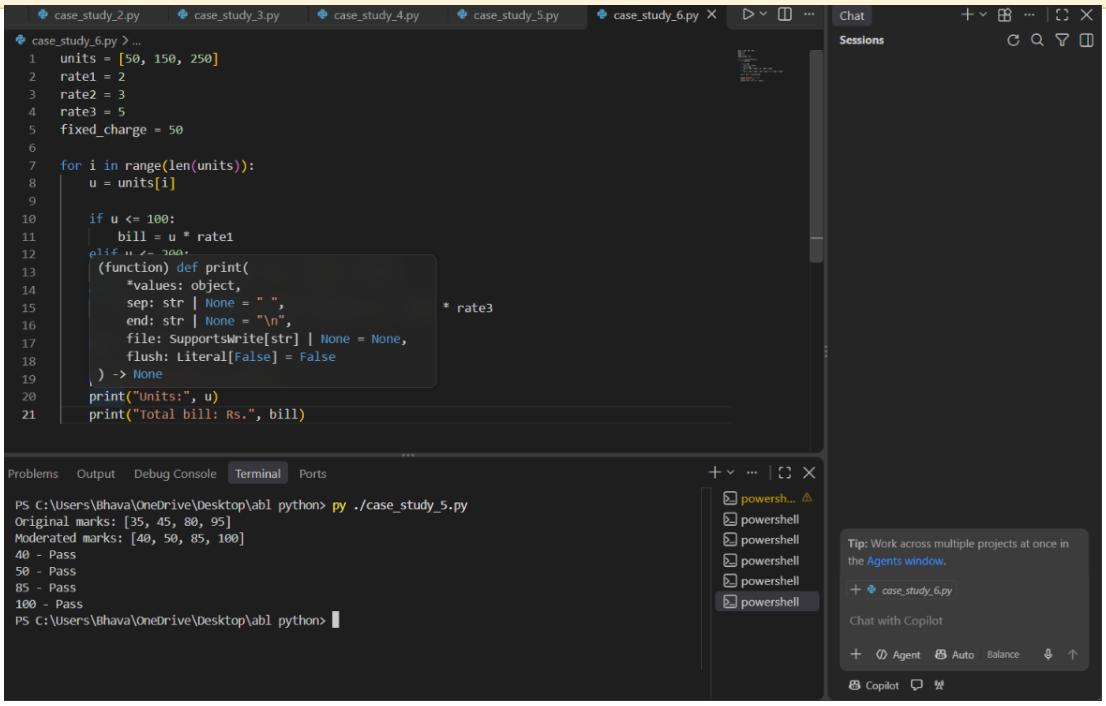
```
BEGIN
  units = apartment consumption values
  rate1 = 2; rate2 = 3; rate3 = 5
  fixed_charge = 50

  FOR each apartment usage u
    IF u <= 100
      bill = u * rate1
    ELSE IF u <= 200
      bill = 100*rate1 + (u-100)*rate2
    ELSE
      bill = 100*rate1 + 100*rate2 + (u-200)*rate3
    END IF
    bill = bill + fixed_charge
    display units and bill
  END FOR
END
```

8. Program and Output Screenshot

The screenshot contains code for apartment electricity billing, but its terminal panel shows output from a previous marks-moderation program. The visible billing code lists unit readings of 50, 150, and 250 units; the terminal values shown should not be treated as electricity-bill results.

> Using the tariff assumptions above, the illustrative bills would be Rs. 150, Rs. 400, and Rs. 800, respectively. These are calculated from the visible rate values and fixed charge, not copied from the screenshot output.



9. Conclusion

A tiered billing calculation can summarize charges consistently across several apartments. For the sample usages, the illustrative progressive-tier bills are Rs. 150 for 50 units, Rs. 400 for 150 units, and Rs. 800 for 250 units, including the fixed charge. The source screenshot's terminal output is unrelated, so it should be replaced with a fresh run of the billing program for a matching code-and-output record.

SMART HOSTEL RESOURCE MONITORING

Python Programming Case Study

1. Case Study

This case study monitors electricity and water usage across hostel floors. The program stores one reading of each resource per floor, displays the readings, and prints a warning if either resource exceeds its configured limit.

2. Problem Statement

Hostel administrators need to notice unusually high resource use so they can investigate possible waste or operational issues. Checking floor-by-floor readings manually can be inconsistent. A program can apply the same limits to each floor and highlight readings that cross them.

3. Objective

To display electricity and water usage for each floor and identify floors where electricity exceeds 150 units or water exceeds 700 units.

4. Assumptions

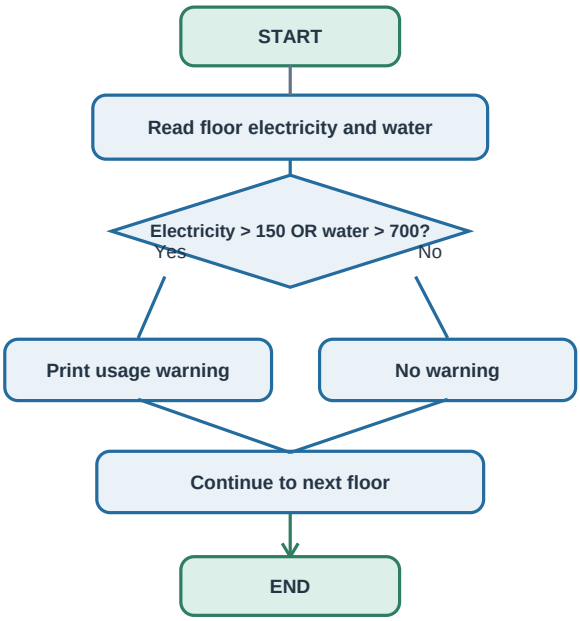
• Electricity and water lists have the same number of entries, with matching indexes for each floor.
• Readings are non-negative and measured over the same reporting period.
• Electricity and water are measured in consistent units across floors.
• The limits are strict: readings equal to 150 electricity units or 700 water units do not trigger the warning.
• Exceeding either limit is sufficient to print a single usage warning for that floor.

5. Algorithm

1. Store electricity and water readings in parallel lists.
2. Loop through the floor indexes.
3. Display the floor number and both resource readings.
4. If electricity is greater than 150 OR water is greater than 700, print “Usage exceeds limit.”
5. Continue until all floors have been processed.

6. Flowchart

Each floor’s readings are checked against both limits; exceeding either one produces a warning.



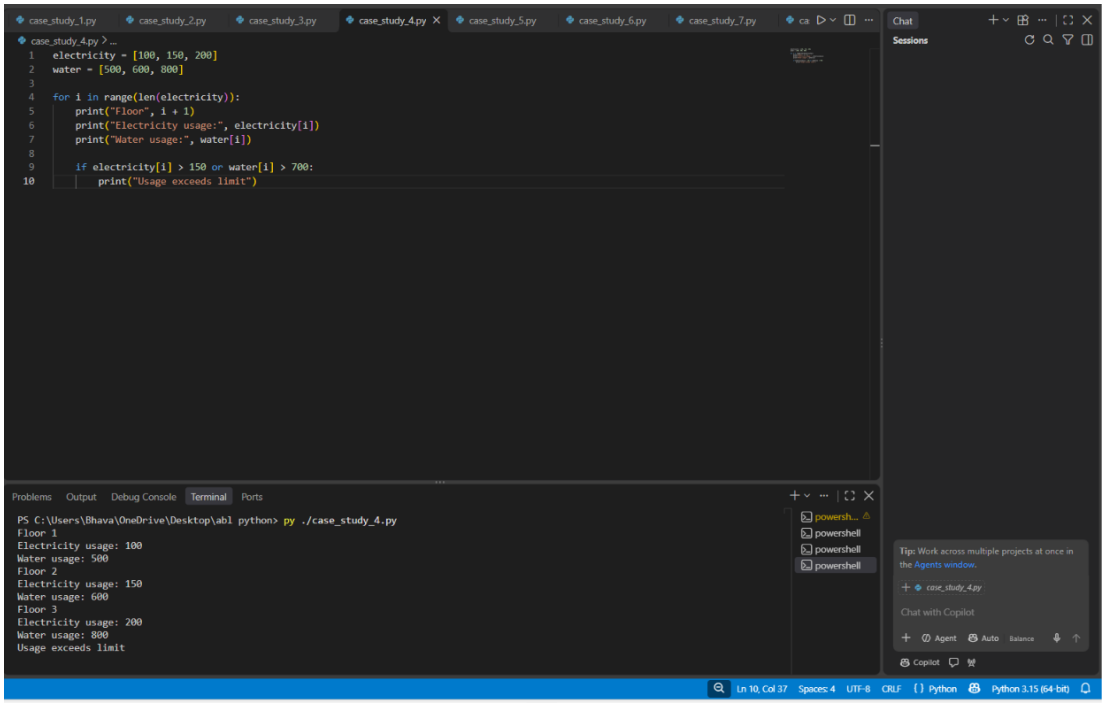
7. Pseudocode

```
BEGIN
  electricity = readings by floor
  water = readings by floor

  FOR each floor index i
    display floor number
    display electricity[i] and water[i]
    IF electricity[i] > 150 OR water[i] > 700
      display usage exceeds limit
    END IF
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows Floor 1 using 100 electricity units and 500 water units, and Floor 2 using 150 and 600; neither triggers a warning. Floor 3 uses 200 electricity units and 800 water units, so the program prints “Usage exceeds limit.”



9. Conclusion

The program checks two resource measures for every floor and flags values above the configured limits. In the sample, only Floor 3 exceeds a limit, with both electricity and water readings above their thresholds. This simple monitoring step can help hostel staff decide where to review resource use in more detail.

STUDENT ACADEMIC RISK CLASSIFICATION

Python Programming Case Study

1. Case Study

This case study demonstrates a rule-based way to classify student records as High Risk, Medium Risk, or Low Risk using marks, attendance, and lab scores. The program checks each student's values against fixed thresholds and prints one category.

2. Problem Statement

Reviewing several academic indicators for each student can take time and may lead to inconsistent screening. A simple program can apply the same stated rules to every record and highlight students whose scores meet the configured risk conditions.

3. Objective

To process aligned lists of marks, attendance, and lab scores and assign each student a risk category using conditional statements.

4. Assumptions

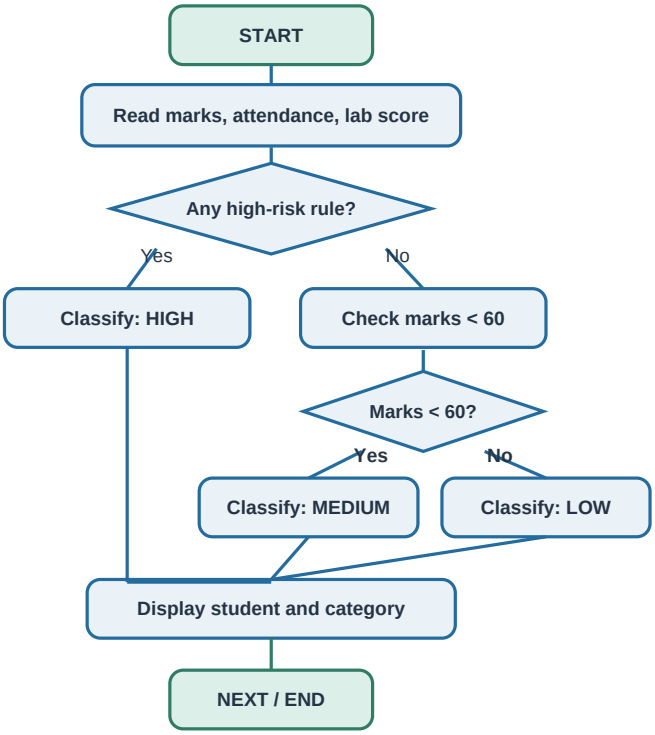
• The marks, attendance, and lab-score lists contain the same number of entries and refer to the same students by index.
• Scores and attendance are numeric and use a 0 to 100 scale.
• High Risk applies if marks are below 40, attendance is below 75, or lab score is below 40.
• If no High Risk condition applies, marks below 60 result in Medium Risk; otherwise the label is Low Risk.
• These labels reflect only the program's sample rules and are not a complete academic assessment.

5. Algorithm

1. Store the marks, attendance, and lab scores in parallel lists.
2. Loop through each student index.
3. If marks < 40 OR attendance < 75 OR lab score < 40, print High Risk.
4. Otherwise, if marks < 60, print Medium Risk.
5. Otherwise, print Low Risk.
6. Continue until every student has been classified.

6. Flowchart

The program first checks the combined High Risk rules, then checks the mark threshold for Medium Risk.

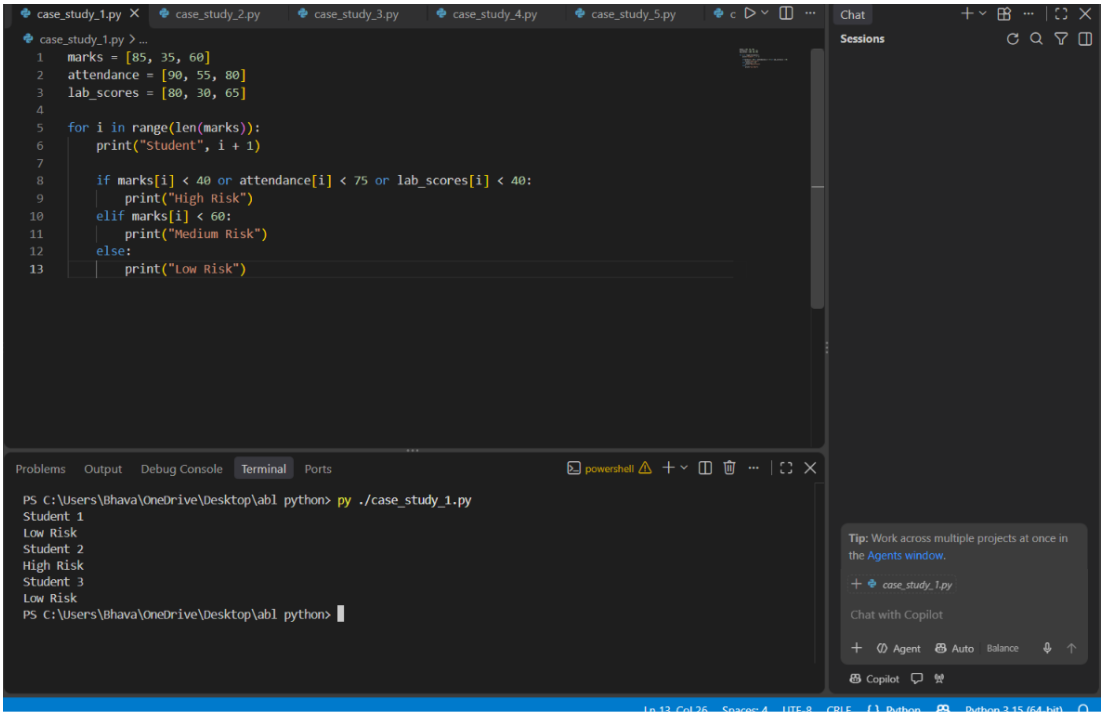


7. Pseudocode

```
BEGIN
  FOR each student index i
    IF marks[i] < 40 OR attendance[i] < 75
      OR lab_scores[i] < 40
      display student and HIGH RISK
    ELSE IF marks[i] < 60
      display student and MEDIUM RISK
    ELSE
      display student and LOW RISK
    END IF
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows Student 1 classified as Low Risk, Student 2 as High Risk, and Student 3 as Low Risk. Student 2 meets multiple High Risk conditions, including marks below 40, attendance below 75, and lab score below 40.



9. Conclusion

The program applies the specified thresholds consistently to each student's marks, attendance, and lab score. In the sample, only Student 2 is classified as High Risk; Students 1 and 3 are classified as Low Risk. The output can support an initial review, while decisions about student support should consider broader context and human judgment.

UNIVERSITY EXAMINATION RESULT MODERATION

Python Programming Case Study

1. Case Study

This case study demonstrates a simple examination result moderation procedure. The program adds 5 marks to each original mark, caps any adjusted mark at 100, stores the moderated marks, and labels each result Pass or Fail using a threshold of 40.

2. Problem Statement

After an approved moderation policy is applied, examination marks must be adjusted consistently and kept within the valid maximum score. Each resulting mark also needs to be classified against the stated pass threshold.

3. Objective

To apply a fixed +5 adjustment to each mark, cap moderated marks at 100, and classify each moderated result as Pass when it is at least 40, otherwise Fail.

4. Assumptions

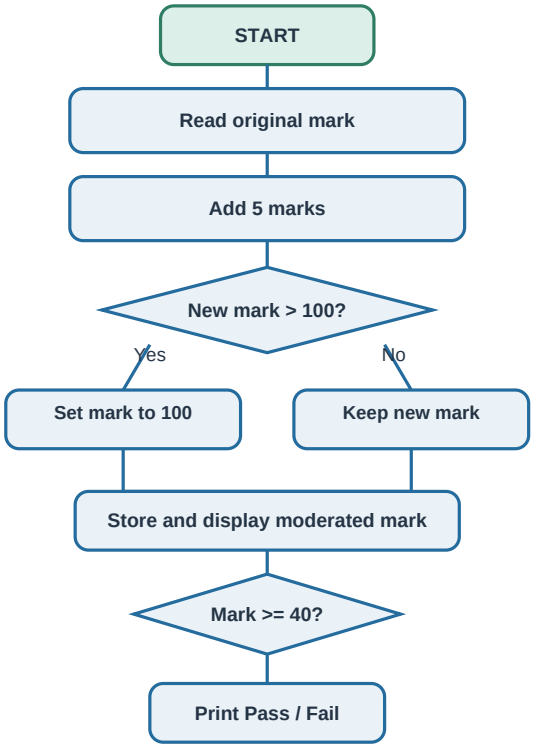
• The mark adjustment of +5 is an authorized example policy for this exercise.
• Original marks are numeric values on a 0 to 100 scale.
• Moderated marks cannot exceed 100; no lower-bound adjustment is applied.
• A moderated mark of 40 or higher is a Pass; below 40 is a Fail.
• The output is a demonstration of the supplied rules and does not establish an actual university policy.

5. Algorithm

1. Store the original marks and create an empty list for moderated marks.
2. For each original mark, add 5.
3. If the adjusted mark exceeds 100, replace it with 100.
4. Add the moderated mark to the result list.
5. Display the original and moderated lists.
6. For each moderated mark, display Pass if it is at least 40; otherwise display Fail.

6. Flowchart

The program adjusts and caps each mark before applying the Pass/Fail threshold.

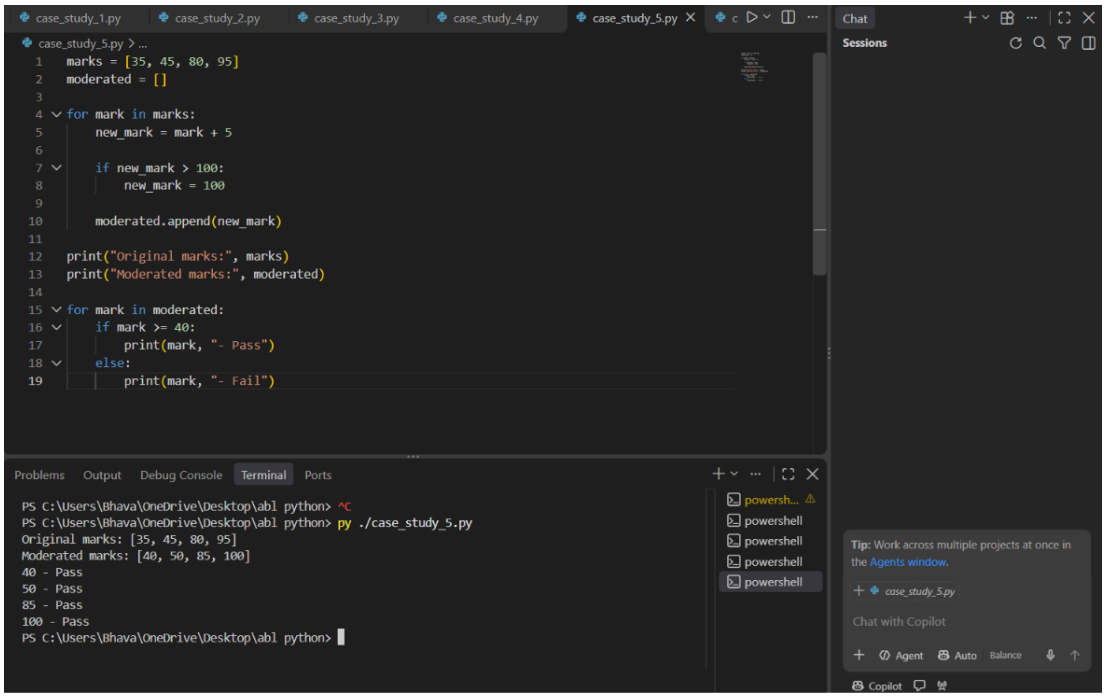


7. Pseudocode

```
BEGIN
  marks = original marks
  moderated = empty list
  FOR each mark in marks
    new_mark = mark + 5
    IF new_mark > 100: new_mark = 100
    append new_mark to moderated
  END FOR
  display original and moderated marks
  FOR each mark in moderated
    IF mark >= 40: display Pass
    ELSE: display Fail
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows original marks [35, 45, 80, 95] and moderated marks [40, 50, 85, 100]. The increase is capped at 100, and all four moderated marks meet the pass threshold of 40.



9. Conclusion

The program applies the stated adjustment and maximum consistently, then classifies every moderated mark. In the sample, the resulting marks are 40, 50, 85, and 100, so each is labeled Pass. In practical use, mark moderation rules should be defined and authorized by the relevant institution before results are changed.